



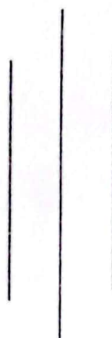
TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING

CHYASAL, LALITPUR



TITLE: comparative study of fcfs, priority & sjf process scheduling algorithms

Report NO: 6

Submitted by:-

Name:- Surajan Shrestha

Roll No:- 38

Submitted To:- Dush mali

Department Of CSIT

Checked by:-

Date of submission:- 2081/05/26

TITLE: Comparative study of fcfs, priority scheduling and sjf process scheduling Algorithms.

OBJECTIVE:

1. To implement three different process scheduling algorithms. fcfs, priority scheduling & sjf.
2. To compute process scheduling metrics such as completion time, TAT, WT & RT.
3. To analyze & compare the performance of these algorithms based on various scheduling metrics.

THEORY:

→ In modern operating systems, CPU scheduling is critical for effective process management. Three common scheduling algorithms are:

• first-come, first-serve (fcfs): The simplest scheduling algorithm, where processes are executed in the order they arrive. This non-preemptive method means that once a process starts, it runs to completion.

Key Concepts:

- Completion Time (CT): The time at which the process finishes.
- Turnaround Time (TAT): $CT - \text{Arrival Time (AT)}$.
- Waiting Time (WT): $TAT - \text{Burst Time (BT)}$.
- Response Time (RT): Time from when the process arrives to when it starts execution.

2. Priority scheduling:- A non-preemptive algorithm where each process has a priority, and the process with the highest priority (lowest numerical value) gets CPU access first. If two processes have the same priority, they are scheduled according to their arrival time.

Key concepts:

- Higher-priority processes are executed first.
- Similar metrics to FCFS are computed (CT, TAT, WT, RT).

3. Shortest Job first (SJF):- A non-preemptive scheduling algorithm where the process with the shortest burst time is executed next. If two processes have the same burst time, the process that arrived first is executed.

Key concepts:

- Shorter burst time processes are executed first.
- SJF minimizes the average waiting time among all processes.

Code Implementation & OBSERVATION:

```
// FCFS
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
struct process {
```

```
int id;
```

```
int bt;
```

```
int at;
```

```
int ct;
```

```
int tat;
```

```
int wt;
```

```
int rt;
```

```
};
```

```
void displayTable (vector<process>& processes) {
```

```
cout << "\n process \t AT \t BT \t CT \t TAT \t WT \t RT \n";
```

```
cout << " - - - - - \n";
```

```
for (const auto& p : processes) {
```

```
cout << "p" << p.id << "\t"
```

```
    << p.at << "\t"
```

```
    << p.bt << "\t"
```

```
    << p.ct << "\t"
```

```
    << p.tat << "\t"
```

```
    << p.wt << "\t"
```

```
    << p.rt << "\n";
```

```
}
```

```
}
```

```
void calculateTimes (vector<process>& processes) {
```

```
int currentTime = 0;
```

```
sort (processes.begin(), processes.end(), [](process& a,
```

```
process& b) { return a.at < b.at; });
```

```
for (auto& p : processes) {
```

```
    currentTime = max (currentTime, p.at);
```

```
    p.rt = currentTime - p.at;
```

$p.ct = \text{currentTime} + p.bt;$

$p.tot = p.ct - p.at;$

$p.wt = p.tot - p.bt;$

$\text{currentTime} = p.ct;$

}

int main() { int n;

cout << "Enter the number of processes %";

cin >> n;

vector<process> processes(n);

for (int i = 0; i < n; i++) {

processes[i].id = i + 1;

cout << "Enter Arrival Time and Burst Time for

process" << i + 1 << " : \n";

cout << "Arrival Time (AT) %";

cin >> processes[i].at;

cout << "Burst Time (BT) %";

cin >> processes[i].bt;

}

calculateTimes(processes);

displayTable(processes);

return 0;

}

OUTPUT :

Enter the number of processes : 3

Enter Arrival Time and Burst Time for process 1 :

Arrival Time (AT) : 0

Burst Time (BT) : 5

Enter Arrival Time and Burst Time for process 1:

Arrival Time (AT): 0

Burst Time (BT): 28

Enter Arrival Time and Burst Time for process 3:

Arrival Time (AT): 0

Burst Time (BT): 18

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	0	28	33	33	5	5
P3	0	18	31	51	33	33

// PRIORITY

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
struct process{
```

```
    int id;
```

```
    int bt;
```

```
    int at;
```

```
    int priority;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
    int rt;
```

```
    bool completed;
```

```
};
```

```
void displayTable (vector<process>& processes){  
    cout << " \n process \t AT \t BT \t AT \t priority \t CT \t TAT \t  
    WT \t RT \n";
```

```
cout << " - - - - - \n";
```

```
for (const auto& p : processes) {
```

```
    cout << "p " << p.id << "\t"
        << p.at << "\t"
        << p.bt << "\t"
        << p.priority << "\t \t"
        << p.ct << "\t"
        << p.tat << "\t"
        << p.wt << "\t"
        << p.rt << "\n";
```

```
    }
```

```
void calculateTimes (vector<process& processes) {
```

```
    int n = processes.size();
```

```
    int currentTime = 0, completedProcesses = 0;
```

```
    sort (processes.begin(), processes.end(), [](process& a,
        process& b) { return a.at < b.at; });
```

```
    while (completedProcesses < n) {
```

```
        int highestPriorityIndex = -1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (!processes[i].completed && processes[i].at <=
                currentTime) {
```

```
                if (highestPriorityIndex == -1 || processes[i].priority
                    < processes[highestPriorityIndex].priority) {
```

```
                    highestPriorityIndex = i;
```

```
                }
```

```
            }
```

```
        }
```

```
if (highestPriorityIndex != -1) {
```

```
processes p = processes[highestPriorityIndex];
```

```
p.at = currentTime - p.at;
```

```
p.ct = currentTime + p.bt;
```

```
p.lat = p.ct - p.at;
```

```
p.wt = p.lat - p.bt;
```

```
p.completed = true;
```

```
completedProcesses++;
```

```
currentTime = p.ct;
```

```
} else { currentTime++; }
```

```
}
```

```
}
```

```
int main() {
```

```
int n;
```

```
cout << "Enter the number of processes : " ;
```

```
cin >> n;
```

```
vector<process> processes(n);
```

```
for (int i = 0; i < n; i++) {
```

```
processes[i].id = i + 1;
```

```
cout << "Enter Arrival Time, Burst Time, and priority
```

```
for process" << i + 1 << " : \n" ;
```

```
cout << "Arrival Time (AT) : " ;
```

```
cin >> processes[i].at ;
```

```
cout << "Burst Time (BT) : " ;
```

```
cin >> processes[i].bt ;
```

```
cout << "priority : " ;
```

```
cin >> processes[i].priority ;
```

```
processes [i].Completed = false ;
```

```
calculateTimes (processes);
```

```
displayTable (processes);
```

```
return 0;
```

```
}
```

OUTPUT :

Enter the number of processes : 3

Enter Arrival Time, Burst Time and priority
for process 1 :

Arrival Time (AT) : 0

Burst Time (BT) : 23

Priority : 3

Enter Arrival Time, Burst Time, and priority for
process 2 :

Arrival Time (AT) : 0

Burst Time (BT) : 10

Priority : 1

Enter Arrival Time, Burst Time and priority
for process 3 :

Arrival Time (AT) : 0

Burst Time (BT) : 45

Priority : 2

process	AT	BT	Priority	CT	TAT	WT	RT
P1	0	23	3	78	78	55	55
P2	0	10	1	10	10	0	0
P3	0	45	2	55	55	10	10

//SJF

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
struct process {
```

```
    int pid;
```

```
    int bt;
```

```
    int at;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
    int rt;
```

```
    bool completed; };
```

```
void displayTable (vector<process>& processes) {
```

```
    cout << " | Process | AT | BT | CT | TAT | WT | RT | \n";
```

```
    cout << " - - - - - \n";
```

```
    for (const auto& p : processes) {
```

```
        cout << "P" << p.pid << "\t"
```

```
            << p.at << "\t"
```

```
            << p.bt << "\t"
```

```
            << p.ct << "\t"
```

```
            << p.tat << "\t"
```

```
            << p.wt << "\t"
```

```
            << p.rt << "\t";
```

```
    }
```

```
}
```

```
void calculateTimes (vector<process>& processes) {
```

```
    int n = processes.size();
```

```
    int currentTime = 0, completedProcesses = 0;
```

```

while (completed processes < n) {
    int shortestJobIndex = -1;
    for (int i = 0; i < n; i++) {
        if (!processes[i].Completed && processes[i].at <=
            currentTime) {
            if (shortestJobIndex == -1 || processes[i].bt < processes
                [shortestJobIndex].bt) {
                shortestJobIndex = i;
            }
        }
    }
    if (shortestJobIndex != -1) {
        process& p = processes[shortestJobIndex];
        p.at = currentTime - p.at;
        p.ct = currentTime + p.bt;
        p.tat = p.ct - p.at;
        p.wt = p.tat - p.bt;
        p.Completed = true;
        completed processes++;
        currentTime = p.ct;
    } else {
        currentTime++;
    }
}

int main() {
    int n;
    cout << "Enter the number of processes ";
    cin >> n;
}

```

```

vector< process> processes (n);
for (int i = 0; i < n; i++) {
    processes[i].id = i + 1;
    cout << "Enter Arrival Time and Burst Time for process " << i + 1 << " : \n";
    cout << "Arrival Time (AT) : ";
    cin >> processes[i].at;
    cout << "Burst Time (BT) : ";
    cin >> processes[i].bt;
    processes[i].Completed = false;
}
calculateTimes(processes);
displayTable (processes);
return 0;
}

```

OUTPUT:

```

Enter the number of processes : 3
Enter Arrival Time and Burst Time for process 1 :
Arrival Time (AT) : 8
Burst Time (BT) : 21
Enter Arrival Time and Burst Time for process 2 :
Arrival Time (AT) : 1
Burst Time (BT) : 18
Enter Arrival Time and Burst Time for process 3 :
Arrival Time (AT) : 6
Burst Time (BT) : 44

```

process	AT	BT	CT	TAT	WT	RT
P1	8	21	40	32	11	11
P2	1	18	19	18	0	0
P3	6	44	84	78	34	34

DISCUSSION:

→ The three scheduling algorithms -- fcfs, priority scheduling, and SJF -- demonstrate varied performance based on different system requirements. fcfs, as the simplest approach, executes processes in the order they arrive, which can cause inefficiencies like the convoy effect, where shorter processes are delayed by longer ones. This results in higher waiting and lat for the processes arriving later. In contrast, priority scheduling allows flexibility by giving precedence to processes with higher priorities, leading to faster execution for critical tasks. However, it can cause starvation for low-priority processes if new higher-priority tasks keep arriving, preventing them from ever getting CPU time. SJF on the other hand, is optimal for reducing waiting time as it prioritizes processes with the shortest burst time. However, its efficiency is dependent on knowing the burst times in the advance, which is often ~~not~~ not possible in real world scenarios.

CONCLUSION:

In conclusion, each scheduling algorithm has its strengths & weakness, with fcfs being the simplest but also the most inefficient. priority scheduling offers flexibility but can lead to process starvation, while SJF minimizes waiting time but relies on prior knowledge of burst times.